

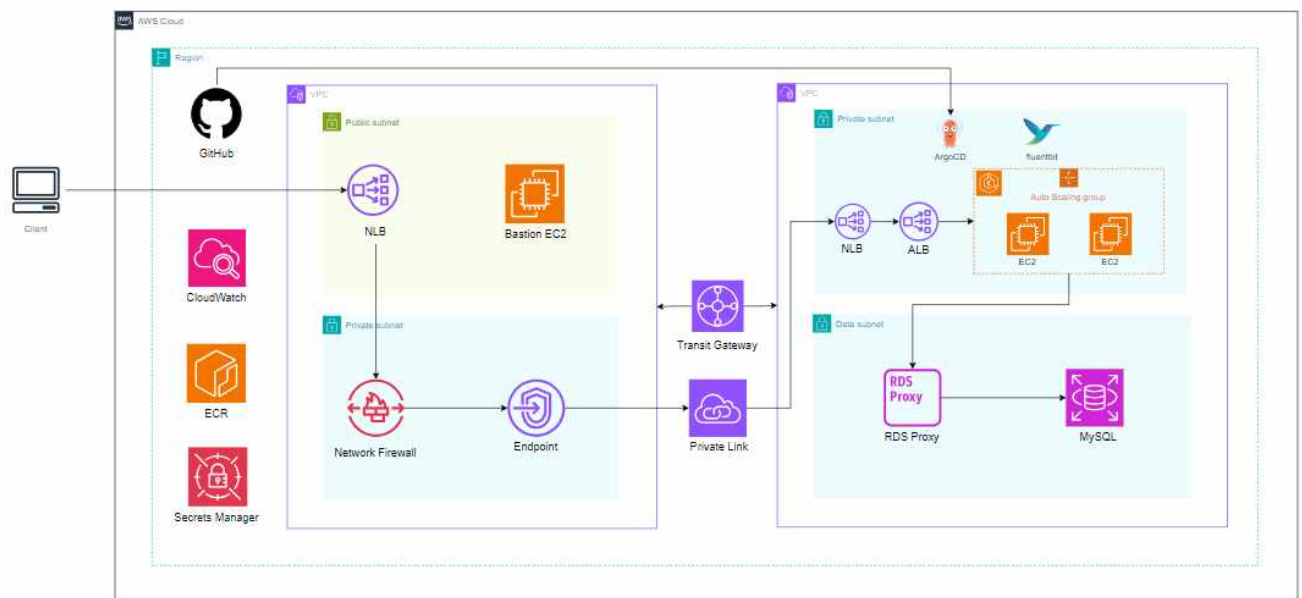
2025년도 전국기능경기대회 과제

직 종 명	클라우드컴퓨팅	과제명	Solution Architecture	과제번호	제1과제
경기시간	4시간	비번호		심사위원 확 인	(인)

1. 요구사항

당신은 Wordskills에서 클라우드 솔루션을 활용하여 어플리케이션이 동작할 수 있는 IT 인프라를 구성하는 업무를 맡고 있습니다. 아래 다이어그램과 주어진 요구사항, 클라우드의 설계원칙인고가용성, 확장성, 비용, 보안 등을 잘 고려하여 인프라를 구축하여야 합니다.

다이어그램



Software Stack

AWS	개발언어/CNCF/Tools
- VPC - EC2 - EKS - ECR - Code series - Secrets Manager - RDS - ALB/NLB - IAM - KMS - Network Firewall - Cloud Watch	- Python - Bash - Helm - ArgoCD - Fluentbit - GitHub

2. 선수 유의사항

※ 다음 유의사항을 고려하여 요구사항을 완성하시오.

- 1) 기계 및 공구 등의 사용 시 안전에 유의하시고, 필요 시 안전장비 및 복장 등을 착용하여 사고를 예방하여 주시기 바랍니다.
- 2) 작업 중 화상, 감전, 찰과상 등 안전사고 예방에 유의하시고, 공구나 작업도구 사용 시 안전 보호구 착용 등 안전수칙을 준수하시기 바랍니다.
- 3) 작업 중 공구의 사용에 주의하고, 안전수칙을 준수하여 사고를 예방하여 주시기 바랍니다.
- 4) 경기 시작 전 가벼운 스트레칭 등으로 긴장을 풀어주시고, 작업도구의 사용 시 안전에 주의하십시오.
- 5) 문제에 제시된 괄호박스 < > 는 변수를 뜻함으로 선수가 적절히 변경하여 사용해야 합니다.
- 6) 문제 풀이와 채점의 효율을 위해 Security Group의 80/443 Outbound는 Anyopen하여 사용할 수 있도록 합니다.
- 7) Bastion EC2는 채점 시 사용되기 때문에 종료되거나 연결 문제, 권한 문제 등으로 발생할 수 있는 불이익을 받지 않도록 주의하시기를 바랍니다.
- 8) 모든 리소스는 서울(ap-northeast-2) 리전에 구성합니다.
- 9) 제공자료는 수정 없이 사용합니다. 제공자료를 수정해서 사용하면 불이익을 받을 수 있습니다.
- 10) 문제에서 주어지지 않는 값들은 AWS Well-Architected Framework 6 pillars를 기준으로 적절한 값을 설정해야 합니다.
- 11) 불필요한 리소스를 생성한 경우, 감점의 요인이 될 수 있습니다. (e.g. VPC 추가 생성)
- 12) 모든 리소스의 이름, 태그, 변수과 변수는 대소문자를 구분합니다.
- 13) 1페이지의 다이어그램은 구성을 추상적으로 표현한 그림으로, 세부적인 구성은 아래의 요구사항을 만족시킬 수 있도록 합니다. (ex. 서브넷이 2개 이상 존재할 수 있습니다.)

3. Network Configuration

- Reference01의 표를 참고하여 2개의 VPC를 생성합니다.
- Hub VPC에는 IGW와 연결되는 Public subnet과 Network Firewall이 위치한 Firewall subnet이 있습니다.
- Application VPC에는 NAT G/W와 연결되는 Private Subnet과 인터넷 통신이 불가능한 Data Subnet이 존재합니다.

4. Transit Gateway

- Hub VPC와 Application VPC가 서로 통신하기 위해서 Transit Gateway를 사용하도록 합니다.
- Transit Gateway Name은 gj2025-tgw으로 지정합니다.
- hub-tgw-attachment Name: gj2025-hub-tgw-attach
- app-tgw-attachment Name: gj2025-app-tgw-attach
- hub-tgw-rtb Name: gj2025-hub-tgw-rtb
- app-tgw-rtb Name: gj2025-app-tgw-rtb

5. Bastion

- 채점을 위해 AWS EC2를 사용하여 Bastion Server를 생성합니다.
- 채점 중 ip가 갑작스럽게 변경되는 상황이 없게끔 재시작시에도 ip는 변경되지 않게 구성하여야 합니다.
- SSH 접근 시 Security을 고려하여 Port Number를 2222로 변경하여야 합니다.
- gj2025-hub-public-subnet-a Subnet에서 실행되어야 합니다.
- Bastion Server는 채점을 위해서 사용되며 어떠한 권한 문제도 없도록 구성해야 합니다.
- Instance Name Tag : gj2025-bastion
- Machine Image : Amazon Linux 2023
- Instance Type : t3.medium
- Required Package : awscli, curl, eksctl, kubectl, jq, argo rollouts CLI(kubectl plugin)

6. Application

- Red, Green 총 2개의 Application이 존재합니다.
- Red, Green 앱은 2개의 API를 가지고 있으며 버전에 따라 GET API의 경우 다른 응답을 제공합니다.(Application 세부 사항은 Reference02 참고)
- 앱의 헬스체크는 /health로 이루어집니다.
- 각 앱은 DB 관련된 설정을 리눅스 환경변수로 전달 받습니다.
- 환경변수 값은 Pod에 직접적으로 명시해선 안되며 Secrets Manager에 저장된 값을 External Secrets를 사용해 읽어와야 합니다.
- DB_USER, DB_PASSWORD, DB_URL이 필요하며 -h 옵션으로 바이너리를 실행하면 도움말을 볼 수 있습니다.

7. Security

- 트래픽 검사를 위해 AWS Network Firewall을 사용합니다.
- Network Firewall은 들어오고 나가는 모든 트래픽을 검사해야 합니다.
- 채점을 위해 Bastion은 모든 연결이 허용되도록 Network Firewall을 구성합니다.
- gj2025-app-vpc에 있는 시스템에서 외부 ifconfig.io로 향하는 HTTP/HTTPS 요청을 차단해야 합니다.
- Network Firewall에 대한 로깅을 활성화 합니다.
- FLOW 유형의 로그만 수집을 하며, Amazon CloudWatch Logs에 전송해야 합니다.
- 규칙은 Stateful 형태의 Suricata rule으로 구성해야 합니다.

- network firewall name: gj2025-firewall
- network firewall policies: gj2025-firewall-policy
- network firewall rule group: gj2025-firewall-rule

8. RDBMS

- Green, Red 앱에서 사용하는 RDBMS 엔진으로 MySQL을 사용하며 데이터베이스 이름은 day1입니다.
- 테이블 생성은 첨부파일 day1_table_v1.sql을 이용합니다.
- 데이터 베이스는 인터넷 접근이 불가능한 서브넷에 배치되어야 합니다.
- 데이터베이스는 Security를 고려하여 포트번호 3309번을 사용하도록 구성 해야 합니다.
- 일반로그, 에러로그, 감사로그 수집은 활성화 해야 합니다.
- 데이터 베이스는 고객관리형키(CMK)를 사용한 암호화를 구성해야 한다.
- 데이터 베이스는 실수로 삭제되는 것을 방지해야 합니다.

- 데이터베이스 인스턴스는 RDS 프록시를 통해서만 트래픽을 수신해야 합니다.

- DB Instance Name: gj2025-db-instance
- Master username: admin
- Master password: Skills53#\$%
- DB instance class: db.t3.medium
- RDS Proxy Name: gj2025-rds-proxy

9. Container Registry

- Registry 컨테이너 이미지 저장을 위해 ECR을 사용하도록 합니다.
- 이미지 저장을 위해 두 개의 ECR repo를 사용합니다.
- 레포지토리는 AWS에서 제공하는 관리형 키 서비스를 통해 암호화되도록 구성해야 합니다.

- Red Repository Name: red
- Green Repository Name: green

10. Container

- 제공된 Application을 Container 환경에 배포하기 위해 AWS EKS를 사용합니다.
- 고가용성을 고려해야 하며, Private 환경에서 애플리케이션이 실행되어야 합니다.
- 컨트롤 플레인에 모든 로그를 수집해야 합니다.
- = 주어진 Application들은 skills 라는 Namespace를 사용하여 EKS Cluster 내에서 논리적으로 분리시켜야 합니다.
- 무중단 배포를 고려하여 구성해야 합니다.
- Daemonset이외에 모든 Addon은 Addon NodeGroup에 배포해야 합니다.
- ExternalSecret 오브젝트 이름은 db-secret으로 설정하고 skills namespace에 배포되어야 합니다. ‘
- ExternalSecret을 위한 IRSA는 ExternalSecret에 권한 인증을 위해서만 사용해야 합니다.

- EKS Cluster Name: gj2025-eks-cluster
- EKS Cluster Version: 1.32

Addon Nodegroup

- 애플리케이션을 제외한 모든 Addon들은 반드시 Addon Nodegroup에서 운용해야 합니다.
- 최소 2개 이상 운용해야 합니다.

- Nodegroup Name: gj2025-eks-addon-nodegroup
- Node EC2 Instance Tag : Name=gj2025-eks-addon-node
- Node EC2 Instance Type: t3.medium

App Nodegroup

- App Nodegroup에서는 애플리케이션을 운용해야 합니다.
- 애플리케이션을 제외한 다른 리소스들은 App Nodegroup에 존재해서는 안됩니다.
- 최소 2개 이상 운용해야 합니다.

- Nodegroup Name: gj2025-eks-app-nodegroup
- Node EC2 Instance Tag: Name=gj2025-eks-app-node
- Node EC2 Instance Type: t3.medium
- Red App Rollout Name: red-rollout
- Green App Rollout Name: green-rollout

11. Secret management

- 애플리케이션 실행을 위한 모든 환경변수는 Secrets Manager를 사용해 저장해야 합니다.
- 또한 Secrets Manager 접근은 IAM 정책 수준에서 최소 보안 준수 원칙을 따라야 합니다.
- GitHub AccessToken의 경우 Secrets Manager를 사용해 저장되어야 합니다.
- External Secrets는 Secrets Manager에 접근하기 위한 권한 인증으로 IRSA(IAM Roles for Service Accounts)를 사용해야 합니다.
- Secret Store CSI Driver를 사용해 Secrets Manager의 값을 마운트해야 합니다.
- Application Secrets Manger Name: gj2025-eks-cluster-catalog-secret
- GitHub AccessToken Secrets Manager Name: gj2025-github-token

12. Logging

- 애플리케이션 로그 기록을 위해 FluentBit을 사용해 구성해야 합니다.
- Fluent-bit는 Daemonset으로 배포되어야 합니다.
- Fluent-bit는 수집한 로그들을 CloudWatch Log Group으로 전송해야 합니다.
- Cloudwatch Logs에서 3초 내로 Fluent-bit에서 전송하는 로그를 확인할 수 있어야 합니다.
- 애플리케이션 로그 중 HTTP Method가 포함된 로그만 CloudWatch Log Group으로 전송해야 합니다.
- 단, /health 경로를 포함한 로그는 제외되어야 합니다.

- Daemonset의 이름은 각각 red-fluent-bit, green-fluent-bit로 구성합니다.
- Fluent-bit는 amazon-cloudwatch Namespace에 구성되어야 합니다.

- Network Firewall Log Group Name : /gj2025/firewall
- Green Application Log Group Name : /gj2025/app/green
- Green Application Log Stream Name : app-green-logs
- Red Application Log Group Name: /gj2025/app/red
- Red Application Log Stream Name: app-red-logs
- CodeBuild Red Log Group Name: /gj2025/build/red
- CodeBuild Green Log Group Name: /gj2025/build/green

13. Load Balancing

- Green, Red 앱은 Application VPC에 있는 Internal ALB를 통해 로드밸런싱 됩니다.
- 외부 사용자가 API에 접근할 시 Hub VPC에 있는 External NLB를 통해 접근됩니다.
- External NLB와 Internal ALB간 통신을 구현하기 PrivateLink를 사용합니다.
- Internal NLB Name: gj2025-app-internal-nlb
- External NLB Name: gj2025-app-external-nlb
- Internal ALB Name: gj2025-app-alb

14. Version Control

- Github public repository를 사용해 버전관리와 CI/CD에 사용합니다.
- 한 개의 레포지토리를 생성하며, 각각 애플리케이션 마다 app, gitops 브랜치를 생성해줍니다.
- 각각의 app 브랜치에는 버전을 제외한 바이너리 파일과 (ex: red, green), Dockerfile, buildspec.yaml 3가지만 존재해야 합니다.
- Code Pipeline 소스 작업 외 모든 작업은 GitHub AccessToken을 사용해 자격증명을 하는 방식을 통해 작업을 수행해야합니다.
- gj2025-repository GithubRepository는 /home/ec2-user/ 경로에 위치해야 합니다.
- ec2-user를 사용해 GithubRepository에 git push 할 수 있어야 합니다.
- Repository Name: gj2025-repository

- Red Application app Branch: app-red
- Red Application gitops Branch: gitops-red
- Green Application app Branch: app-green
- Green Application gitops Branch: gitops-green

15. Continuous Intigreation

- CodeBuild와 CodePipeline을 사용해 CI를 구성합니다.
 - Golang 기반의 두 개의 red, green 애플리케이션을 빌드합니다.
 - 로깅을 반드시 활성화 해야 합니다.
 - 수집한 로그는 Amazon CloudWatch Logs에 전송해야 합니다.
 - CodePipeline의 소스 설정을 위해 GitHub OAuth로 자격증명을 인증합니다.
- Red Build Project Name: gj2025-app-red-build
 - Green Build Project Name: gj2025-app-green-build
 - Red Pipeline Name: gj2025-app-red-pipeline
 - Green Pipeline Name: gj2025-app-green-pipeline

16. Continuous Delivery

- 새로운 버전 배포를 위한 파이프라인을 구성해야 합니다.
 - ArgoCD를 사용하여 구성해야 합니다.
 - ArgoCD는 App VPC에 배포되어야 합니다.
 - 대시보드는 Hub VPC에 External NLB를 통하여 접근되도록 구성합니다.
 - admin User로 로그인 되어야 하며 Skills53 Password를 사용해야 합니다.
 - gitops 브랜치에 새로운 커밋이 푸쉬되는 것을 감지하여 Blue/Green 방식으로 롤아웃되어야 합니다.
 - 새로운 커밋이 push 될 경우 새로운 애플리케이션은 Green Stage에서 /health 경로로 Health Check를 진행해야합니다.
 - Health Check 후 애플리케이션에 문제가 없을 시 Blue Stage로 배포되어야 합니다.
 - CI/CD는 3분안에 작업이 완료되어야 합니다.
- External NLB Name: gj2025-argo-external-nlb
 - Internal NLB Name: gj2025-argo-internal-nlb

Reference01

- VPC -

Name	Cidr
gj2025-hub-vpc	10.0.0.0/16
gj2025-app-vpc	192.168.0.0/16

- Subnet -

Name	Cidr	VPC
gj2025-hub-public-subnet-a	10.0.0.0/24	gj2025-hub-vpc
gj2025-hub-public-subnet-b	10.0.1.0/24	gj2025-hub-vpc
gj2025-hub-private-subnet-a	10.0.2.0/24	gj2025-hub-vpc
gj2025-hub-private-subnet-b	10.0.3.0/24	gj2025-hub-vpc
gj2025-hub-firewall-subnet	10.0.4.0/24	gj2025-hub-vpc
gj2025-app-private-subnet-a	192.168.0.0/24	gj2025-app-vpc
gj2025-app-private-subnet-b	192.168.1.0/24	gj2025-app-vpc
gj2025-app-data-subnet-a	192.168.2.0/24	gj2025-app-vpc
gj2025-app-data-subnet-b	192.168.3.0/24	gj2025-app-vpc

- Routing Table -

Name	Subnet	Gateway
gj2025-hub-public-rtb	gj2025-hub-public-subnet-a gj2025-hub-public-subnet-b	Internet Gateway (gj2025-hub-igw)
gj2025-hub-private-rtb-a	gj2025-hub-private-subnet-a	Transit Gateway (gj2025-tgw)
gj2025-hub-private-rtb-b	gj2025-hub-private-subnet-a	Transit Gateway (gj2025-tgw)
gj2025-hub-firewall-rtb	gj2025-firewall-subnet	Nat Gateway (gj2025-hub-ngw)
gj2025-app-private-rtb-a	gj2025-app-private-subnet-a	Transit Gateway (gj2025-tgw)
gj2025-app-private-rtb-b	gj2025-app-private-subnet-a	Transit Gateway (gj2025-tgw)
gj2025-app-data-rtb-a	gj2025-app-data-subnet-b	X
gj2025-app-data-rtb-b	gj2025-app-data-subnet-b	X

Reference02

- Customer -

Verbm, Path	Request	Response (v1.0.0)	Response (v1.0.1)
GET /health	{ "x": "abcd", "y": 21 }	{ "status": "inserted", "id": "xx11abcd" }	
POST /green	?id=xx11abcd	{ "x": "abcd", "y": 21, "version": "1.0.0" }	{ "x", "abcd", "y": 21, "version": "1.0.1" }
GET /green	{ "name": "kim" }	{ "status": "inserted", "id": "yy11abcd" }	
GET /red	?id=yy11abcd	{ "name": "kim", "version": "1.0.0" }	{ "name": "kim", "version": "1.0.1" }